

```

1 import components.set.Set;
2
3
4
5
6
7
8 /**
9  * Utility class to support string reassembly from fragments.
10 *
11 * @author Eric Zhou
12 *
13 * @mathdefinitions <pre>
14 *
15 * OVERLAPS (
16 *   s1: string of character,
17 *   s2: string of character,
18 *   k: integer
19 * ) : boolean is
20 *   0 <= k and k <= |s1| and k <= |s2| and
21 *   s1[|s1|-k, |s1|) = s2[0, k)
22 *
23 * SUBSTRINGS (
24 *   strSet: finite set of string of character,
25 *   s: string of character
26 * ) : finite set of string of character is
27 * {t: string of character
28 *   where (t is in strSet and t is substring of s)
29 *   (t)}
30 *
31 * SUPERSTRINGS (
32 *   strSet: finite set of string of character,
33 *   s: string of character
34 * ) : finite set of string of character is
35 * {t: string of character
36 *   where (t is in strSet and s is substring of t)
37 *   (t)}
38 *
39 * CONTAINS_NO_SUBSTRING_PAIRS (
40 *   strSet: finite set of string of character
41 * ) : boolean is
42 * for all t: string of character
43 *   where (t is in strSet)
44 *   (SUBSTRINGS(strSet \ {t}, t) = {})
45 *
46 * ALL_SUPERSTRINGS (
47 *   strSet: finite set of string of character
48 * ) : set of string of character is
49 * {t: string of character
50 *   where (SUBSTRINGS(strSet, t) = strSet)
51 *   (t)}
52 *
53 * CONTAINS_NO_OVERLAPPING_PAIRS (
54 *   strSet: finite set of string of character
55 * ) : boolean is
56 * for all t1, t2: string of character, k: integer
57 *   where (t1 != t2 and t1 is in strSet and t2 is in strSet and
58 *         1 <= k and k <= |s1| and k <= |s2|)
59 *   (not OVERLAPS(s1, s2, k))
60 *
61 * </pre>
62 */
63 public final class StringReassembly {
64
65     /**
66     * Private no-argument constructor to prevent instantiation of this utility
67     * class.
68     */
69     private StringReassembly() {
70

```

```

71
72 /**
73  * Reports the maximum length of a common suffix of {@code str1} and prefix
74  * of {@code str2}.
75  *
76  * @param str1
77  *     first string
78  * @param str2
79  *     second string
80  * @return maximum overlap between right end of {@code str1} and left end of
81  *     {@code str2}
82  * @requires <pre>
83  * str1 is not substring of str2  and
84  * str2 is not substring of str1
85  * </pre>
86  * @ensures <pre>
87  * OVERLAPS(str1, str2, overlap)  and
88  * for all k: integer
89  *     where (overlap < k  and  k <= |str1|  and  k <= |str2|)
90  *     (not OVERLAPS(str1, str2, k))
91  * </pre>
92  */
93 public static int overlap(String str1, String str2) {
94     assert str1 != null : "Violation of: str1 is not null";
95     assert str2 != null : "Violation of: str2 is not null";
96     assert str2.indexOf(str1) < 0
97           : "Violation of: " + "str1 is not substring of str2";
98     assert str1.indexOf(str2) < 0
99           : "Violation of: " + "str2 is not substring of str1";
100
101     /*
102     * Start with maximum possible overlap and work down until a match is
103     * found; think about it and try it on some examples to see why
104     * iterating in the other direction doesn't work
105     */
106     int maxOverlap = str2.length() - 1;
107     while (!str1.regionMatches(str1.length() - maxOverlap, str2, 0, maxOverlap)) {
108         maxOverlap--;
109     }
110     return maxOverlap;
111 }
112
113 /**
114  * Returns concatenation of {@code str1} and {@code str2} from which one of
115  * the two "copies" of the common string of {@code overlap} characters at
116  * the end of {@code str1} and the beginning of {@code str2} has been
117  * removed.
118  *
119  * @param str1
120  *     first string
121  * @param str2
122  *     second string
123  * @param overlap
124  *     amount of overlap
125  * @return combination with one "copy" of overlap removed
126  * @requires OVERLAPS(str1, str2, overlap)
127  * @ensures combination = str1[0, |str1|-overlap) * str2
128  */
129 public static String combination(String str1, String str2, int overlap) {
130     assert str1 != null : "Violation of: str1 is not null";
131     assert str2 != null : "Violation of: str2 is not null";
132     assert 0 <= overlap && overlap <= str1.length() && overlap <= str2.length()
133           && str1.regionMatches(str1.length() - overlap, str2, 0, overlap)
134           : "" + "Violation of: OVERLAPS(str1, str2, overlap)";
135
136     /*

```

```

136     * Hint: consider using substring (a String method)
137     */
138
139     String result; //remove overlap part in str1
140     result = str1.substring(0, str1.length() - overlap) + str2;
141     return result;
142
143 }
144
145 /**
146  * Adds {@code str} to {@code strSet} if and only if it is not a substring
147  * of any string already in {@code strSet}; and if it is added, also removes
148  * from {@code strSet} any string already in {@code strSet} that is a
149  * substring of {@code str}.
150  *
151  * @param strSet
152  *         set to consider adding to
153  * @param str
154  *         string to consider adding
155  * @updates strSet
156  * @requires CONTAINS_NO_SUBSTRING_PAIRS(strSet)
157  * @ensures <pre>
158  * if SUPERSTRINGS(#strSet, str) = {}
159  * then strSet = #strSet union {str} \ SUBSTRINGS(#strSet, str)
160  * else strSet = #strSet
161  * </pre>
162  */
163 public static void addToSetAvoidingSubstrings(Set<String> strSet, String str) {
164     assert strSet != null : "Violation of: strSet is not null";
165     assert str != null : "Violation of: str is not null";
166     /*
167      * Note: Precondition not checked!
168      */
169
170     /*
171      * Hint: consider using contains (a String method)
172      */
173     boolean result = true;
174     Set<String> temp = strSet.newInstance();
175     while (strSet.size() > 0) {
176         String s = strSet.removeAny(); //choose one element in set
177         if (s.contains(str)) {
178             result = false;
179         }
180         if (!str.contains(s)) {
181             temp.add(s);
182         }
183     }
184     if (result) { //do not contain
185         temp.add(str);
186     }
187     strSet.transferFrom(temp);
188
189 }
190
191 /**
192  * Returns the set of all individual lines read from {@code input}, except
193  * that any line that is a substring of another is not in the returned set.
194  *
195  * @param input
196  *         source of strings, one per line
197  * @return set of lines read from {@code input}
198  * @requires input.is_open
199  * @ensures <pre>
200  * input.is_open and input.content = <> and

```

```

201 * linesFromInput = [maximal set of lines from #input.content such that
202 *                     CONTAINS_NO_SUBSTRING_PAIRS(linesFromInput)]
203 * </pre>
204 */
205 public static Set<String> linesFromInput(SimpleReader input) {
206     assert input != null : "Violation of: input is not null";
207     assert input.isOpen() : "Violation of: input.is_open";
208
209     Set<String> result = new Set1L<>();
210     while (!input.atEOS()) {
211         String line = input.nextLine(); //check every line
212         addToSetAvoidingSubstrings(result, line);
213     }
214
215     return result;
216 }
217
218 /**
219 * Returns the longest overlap between the suffix of one string and the
220 * prefix of another string in {@code strSet}, and identifies the two
221 * strings that achieve that overlap.
222 *
223 * @param strSet
224 *     the set of strings examined
225 * @param bestTwo
226 *     an array containing (upon return) the two strings with the
227 *     largest such overlap between the suffix of {@code bestTwo[0]}
228 *     and the prefix of {@code bestTwo[1]}
229 * @return the amount of overlap between those two strings
230 * @replaces bestTwo[0], bestTwo[1]
231 * @requires <pre>
232 *     CONTAINS_NO_SUBSTRING_PAIRS(strSet) and
233 *     bestTwo.length >= 2
234 * </pre>
235 * @ensures <pre>
236 *     bestTwo[0] is in strSet and
237 *     bestTwo[1] is in strSet and
238 *     OVERLAPS(bestTwo[0], bestTwo[1], bestOverlap) and
239 *     for all str1, str2: string of character, overlap: integer
240 *         where (str1 is in strSet and str2 is in strSet and
241 *             OVERLAPS(str1, str2, overlap))
242 *         (overlap <= bestOverlap)
243 * </pre>
244 */
245 private static int bestOverlap(Set<String> strSet, String[] bestTwo) {
246     assert strSet != null : "Violation of: strSet is not null";
247     assert bestTwo != null : "Violation of: bestTwo is not null";
248     assert bestTwo.length >= 2 : "Violation of: bestTwo.length >= 2";
249     /*
250     * Note: Rest of precondition not checked!
251     */
252     int bestOverlap = 0;
253     Set<String> processed = strSet.newInstance();
254     while (strSet.size() > 0) {
255         /*
256         * Remove one string from strSet to check against all others
257         */
258         String str0 = strSet.removeAny();
259         for (String str1 : strSet) {
260             /*
261             * Check str0 and str1 for overlap first in one order...
262             */
263             int overlapFrom0To1 = overlap(str0, str1);
264             if (overlapFrom0To1 > bestOverlap) {
265                 /*

```

```

265         * Update best overlap found so far, and the two strings
266         * that produced it
267         */
268         bestOverlap = overlapFrom0To1;
269         bestTwo[0] = str0;
270         bestTwo[1] = str1;
271     }
272     /*
273     * ... and then in the other order
274     */
275     int overlapFrom1To0 = overlap(str1, str0);
276     if (overlapFrom1To0 > bestOverlap) {
277         /*
278         * Update best overlap found so far, and the two strings
279         * that produced it
280         */
281         bestOverlap = overlapFrom1To0;
282         bestTwo[0] = str1;
283         bestTwo[1] = str0;
284     }
285 }
286 /*
287 * Record that str0 has been checked against every other string in
288 * strSet
289 */
290 processed.add(str0);
291 }
292 /*
293 * Restore strSet and return best overlap
294 */
295 strSet.transferFrom(processed);
296 return bestOverlap;
297 }
298 }
299
300 /**
301 * Combines strings in {@code strSet} as much as possible, leaving in it
302 * only strings that have no overlap between a suffix of one string and a
303 * prefix of another. Note: uses a "greedy approach" to assembly, hence may
304 * not result in {@code strSet} being as small a set as possible at the end.
305 *
306 * @param strSet
307 *     set of strings
308 * @updates strSet
309 * @requires CONTAINS_NO_SUBSTRING_PAIRS(strSet)
310 * @ensures <pre>
311 * ALL_SUPERSTRINGS(strSet) is subset of ALL_SUPERSTRINGS(#strSet) and
312 * |strSet| <= |#strSet| and
313 * CONTAINS_NO_SUBSTRING_PAIRS(strSet) and
314 * CONTAINS_NO_OVERLAPPING_PAIRS(strSet)
315 * </pre>
316 */
317 public static void assemble(Set<String> strSet) {
318     assert strSet != null : "Violation of: strSet is not null";
319     /*
320     * Note: Precondition not checked!
321     */
322     /*
323     * Combine strings as much possible, being greedy
324     */
325     boolean done = false;
326     while ((strSet.size() > 1) && !done) {
327         String[] bestTwo = new String[2];
328         int bestOverlap = bestOverlap(strSet, bestTwo);
329         if (bestOverlap == 0) {
330             /*

```

```

331     * No overlapping strings remain; can't do any more
332     */
333     done = true;
334 } else {
335     /*
336     * Replace the two most-overlapping strings with their
337     * combination; this can be done with add rather than
338     * addToSetAvoidingSubstrings because the latter would do the
339     * same thing (this claim requires justification)
340     */
341     strSet.remove(bestTwo[0]);
342     strSet.remove(bestTwo[1]);
343     String overlapped = combination(bestTwo[0], bestTwo[1], bestOverlap);
344     strSet.add(overlapped);
345 }
346 }
347 }
348
349 /**
350  * Prints the string {@code text} to {@code out}, replacing each '~' with a
351  * line separator.
352  *
353  * @param text
354  *         string to be output
355  * @param out
356  *         output stream
357  * @updates out
358  * @requires out.is_open
359  * @ensures <pre>
360  *   out.is_open and
361  *   out.content = #out.content *
362  *   [text with each '~' replaced by line separator]
363  * </pre>
364  */
365 public static void printWithLineSeparators(String text, SimpleWriter out) {
366     assert text != null : "Violation of: text is not null";
367     assert out != null : "Violation of: out is not null";
368     assert out.isOpen() : "Violation of: out.is_open";
369
370     out.print(text.replace('~', '\n')); //use "replace" to replace '~'
371 }
372
373
374 /**
375  * Given a file name (relative to the path where the application is running)
376  * that contains fragments of a single original source text, one fragment
377  * per line, outputs to stdout the result of trying to reassemble the
378  * original text from those fragments using a "greedy assembler". The
379  * result, if reassembly is complete, might be the original text; but this
380  * might not happen because a greedy assembler can make a mistake and end up
381  * predicting the fragments were from a string other than the true original
382  * source text. It can also end up with two or more fragments that are
383  * mutually non-overlapping, in which case it outputs the remaining
384  * fragments, appropriately labelled.
385  *
386  * @param args
387  *         Command-line arguments: not used
388  */
389 public static void main(String[] args) {
390     SimpleReader in = new SimpleReader1L();
391     SimpleWriter out = new SimpleWriter1L();
392     /*
393     * Get input file name
394     */
395     out.print("Input file (with fragments): ");

```

```

396 String inputFileName = in.nextLine();
397 SimpleReader inFile = new SimpleReader1L(inputFileName);
398 /*
399  * Get initial fragments from input file
400  */
401 Set<String> fragments = LinesFromInput(inFile);
402 /*
403  * Close inFile; we're done with it
404  */
405 inFile.close();
406 /*
407  * Assemble fragments as far as possible
408  */
409 assemble(fragments);
410 /*
411  * Output fully assembled text or remaining fragments
412  */
413 if (fragments.size() == 1) {
414     out.println();
415     String text = fragments.removeAny();
416     printWithLineSeparators(text, out);
417 } else {
418     int fragmentNumber = 0;
419     for (String str : fragments) {
420         fragmentNumber++;
421         out.println();
422         out.println("-----");
423         out.println("  -- Fragment #" + fragmentNumber + ": --");
424         out.println("-----");
425         printWithLineSeparators(str, out);
426     }
427 }
428 /*
429  * Close input and output streams
430  */
431 in.close();
432 out.close();
433 }
434
435 }
436

```